DevSecOps w teorii i praktyce

Konrad Sagała

Cloud Security Architect, Alior Bank

Trochę o sobie

- Cloud Security Architect – Azure/M365
- Microsoft Certified Trainer since 2007
- Microsoft MVP since 2007 – M365 & Security
- Twitter - @sagus
- LinkedIn - @konradsagala
- Github - https://github.com/ksagala
- Blog – https://pepugmaster.blogspot.com
- Hobby – Podróże, Śpiew, Taniec





Agenda

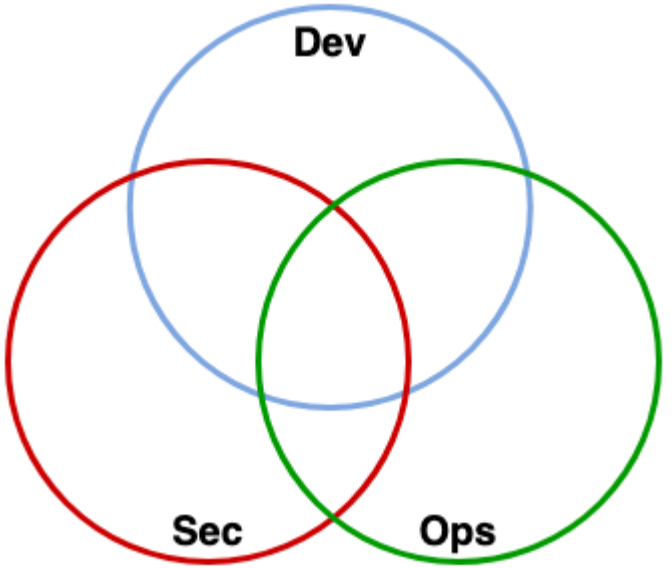
- Understanding DevSecOps and the Need for Security in DevOps
- Importance of Security in the DevOps Lifecycle
- DevSecOps in Multicloud
- DevSecOps Tools
- DevSecOps Best Practices

Understanding DevSecOps and the Need for Security in DevOps

Risks and Challenges in Unsecured DevOps Environments

Vulnerability to Misconfigurations

DevOps pipelines lacking proper security are prone to misconfigurations that expose the environment to threats.



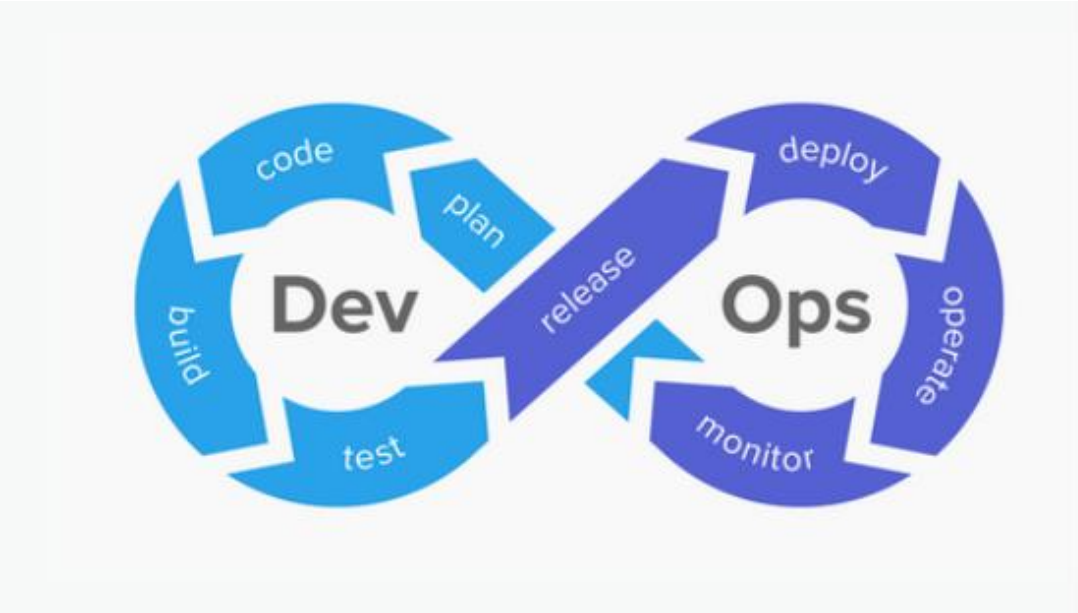
Supply Chain Attacks

Unsecured DevOps environments face risks from supply chain attacks compromising software integrity.

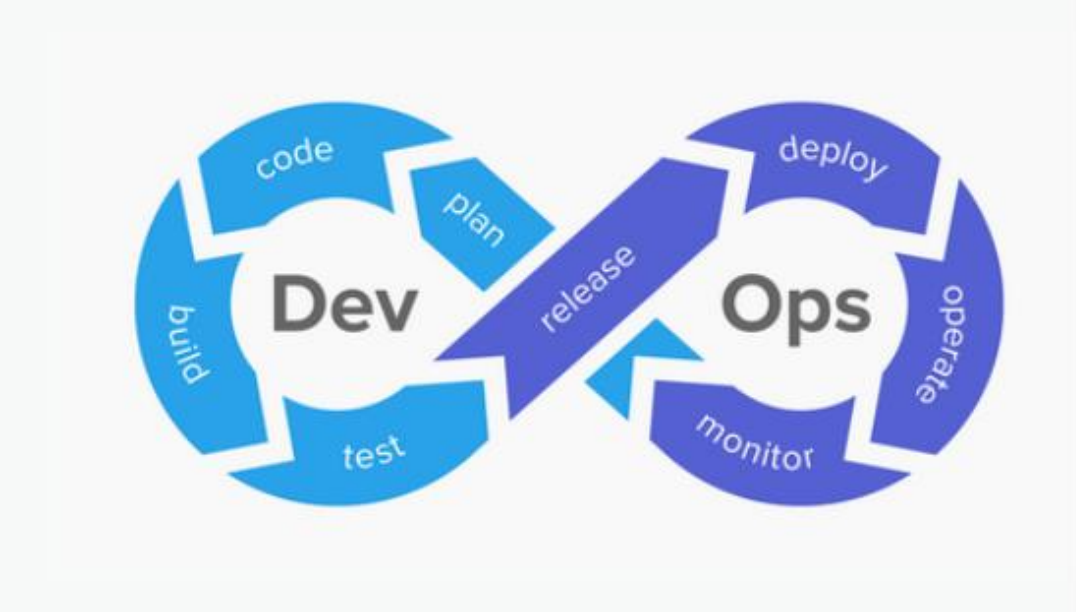
Unauthorized Access Risks

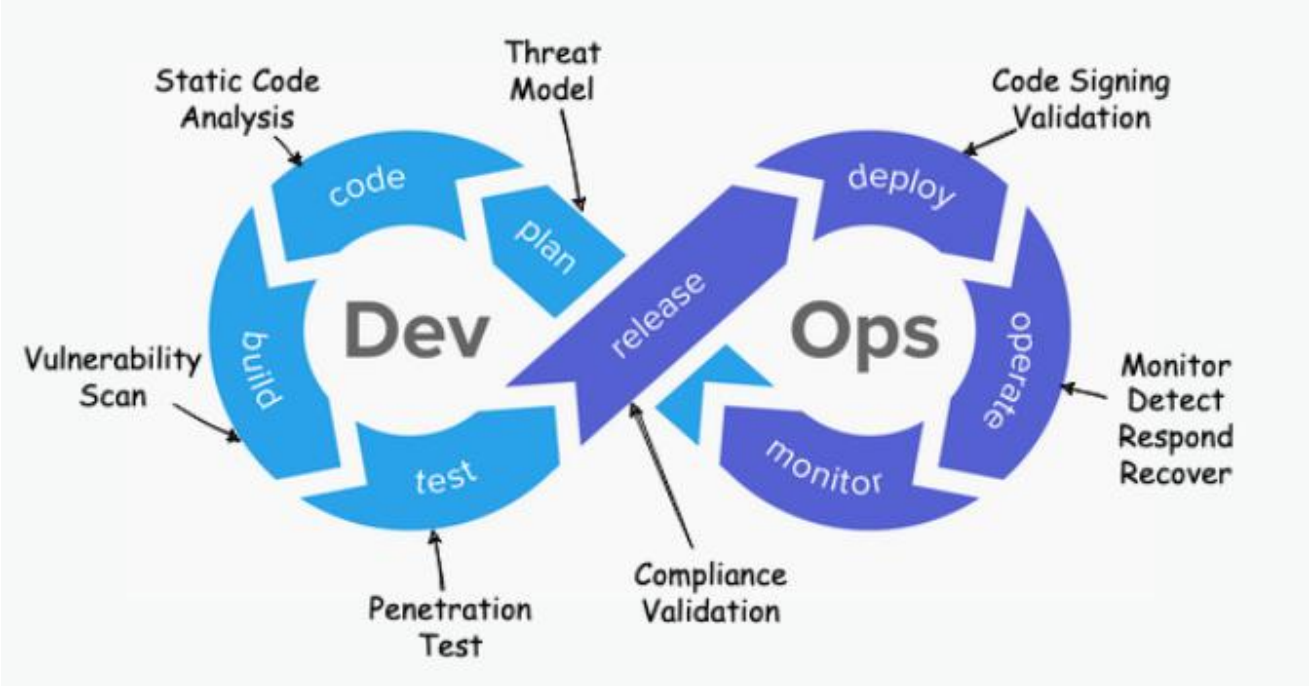
Lack of integrated security leads to unauthorized access, increasing chances of data breaches and compliance failures.

DevOps vs DevSecOps



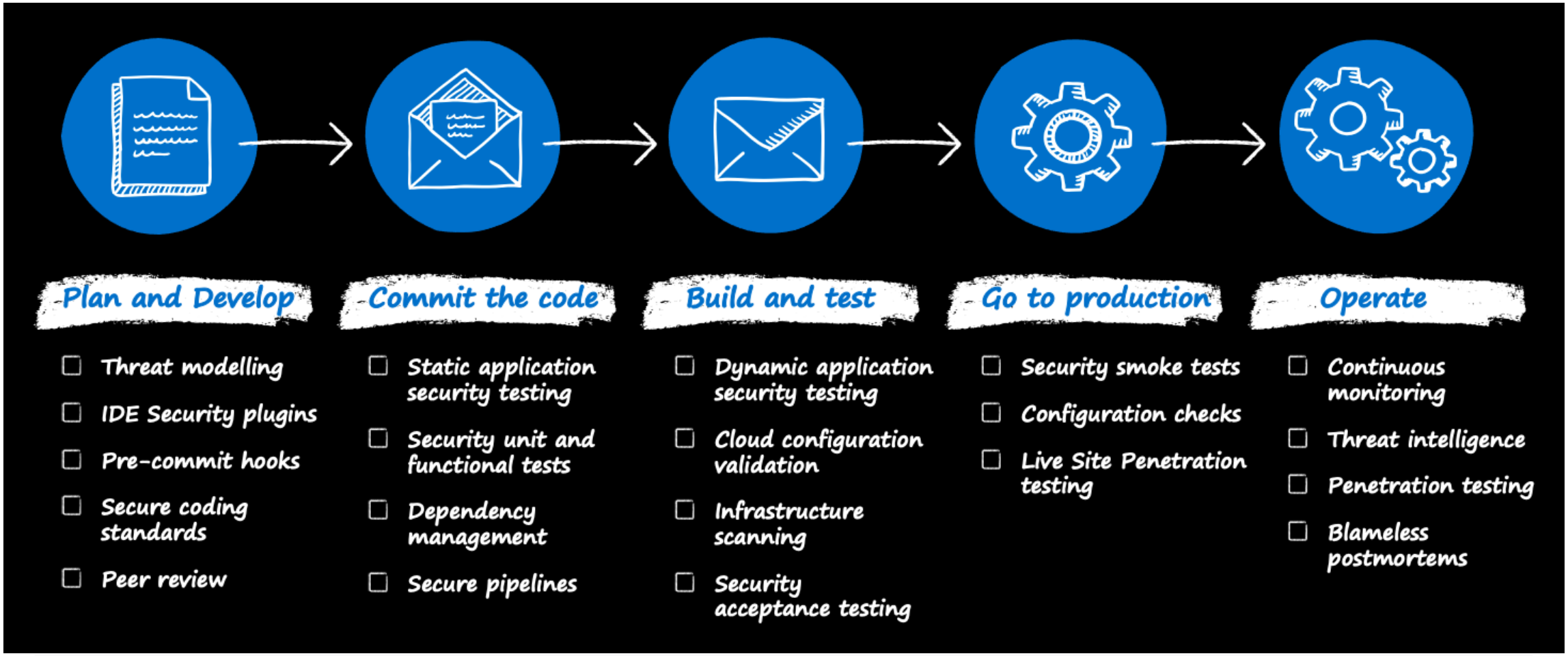
DevOps vs DevSecOps





https://owasp.org/www-project-devsecops-guideline/latest/00a-Overview

High level DevSecOps process



https://learn.microsoft.com/en-us/azure/well-architected/security/secure-development-lifecycle

Importance of Security in the DevOps Lifecycle

Threats Targeting DevOps Pipelines and Toolchains

Code Injection Threats

Code injection attacks introduce malicious code into DevOps pipelines, risking data integrity and system stability.

Compromised Credentials

Stolen or weak credentials enable unauthorized access, threatening pipeline security and operations.

Vulnerable Third-Party Components

Using vulnerable third-party components exposes pipelines to security risks and potential exploits.

Benefits of Integrating Security Throughout the Development Process

Early Vulnerability Reduction

Embedding security early in development minimizes vulnerabilities before they become risks.

Faster Remediation

Early security enables quicker identification and resolution of security issues.

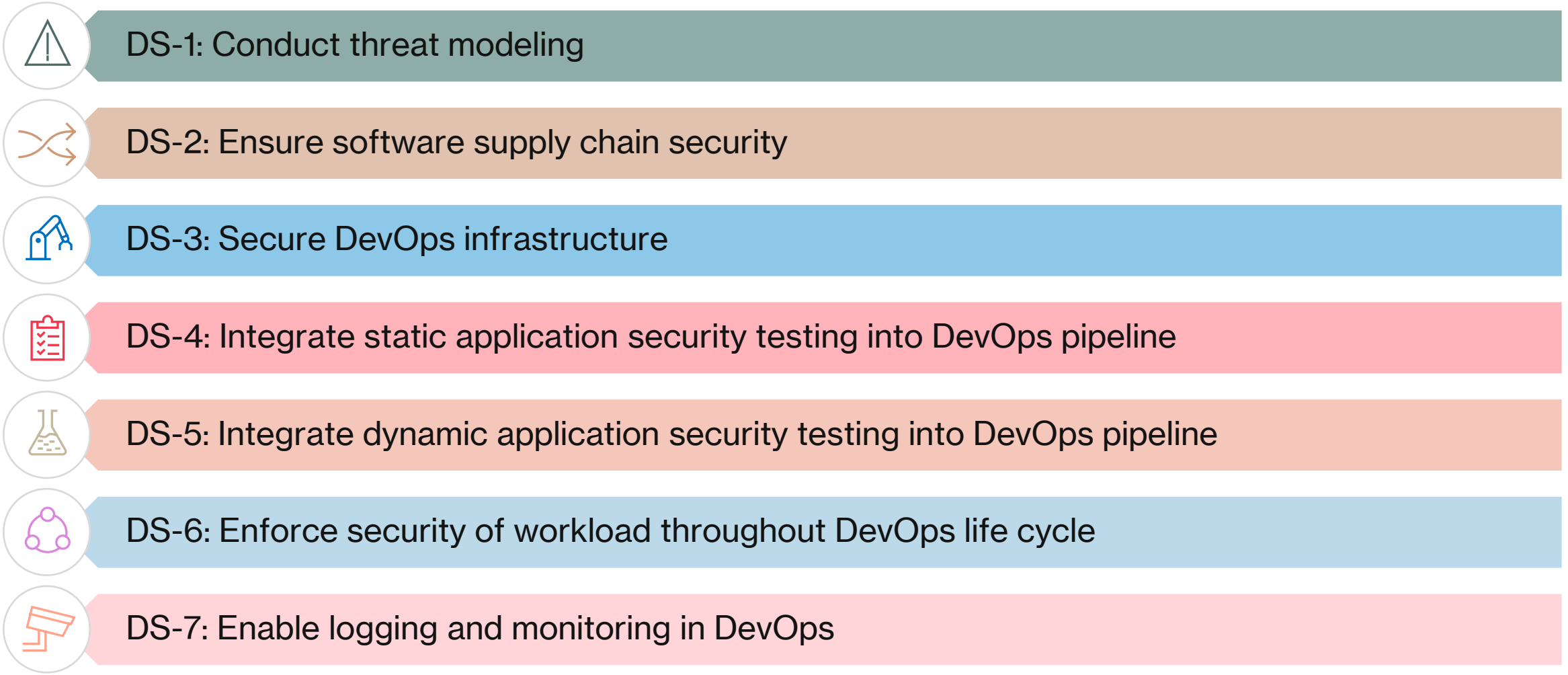
Compliance Assurance

Integrating security ensures adherence to regulatory and industry compliance standards.

Resilient Software Delivery

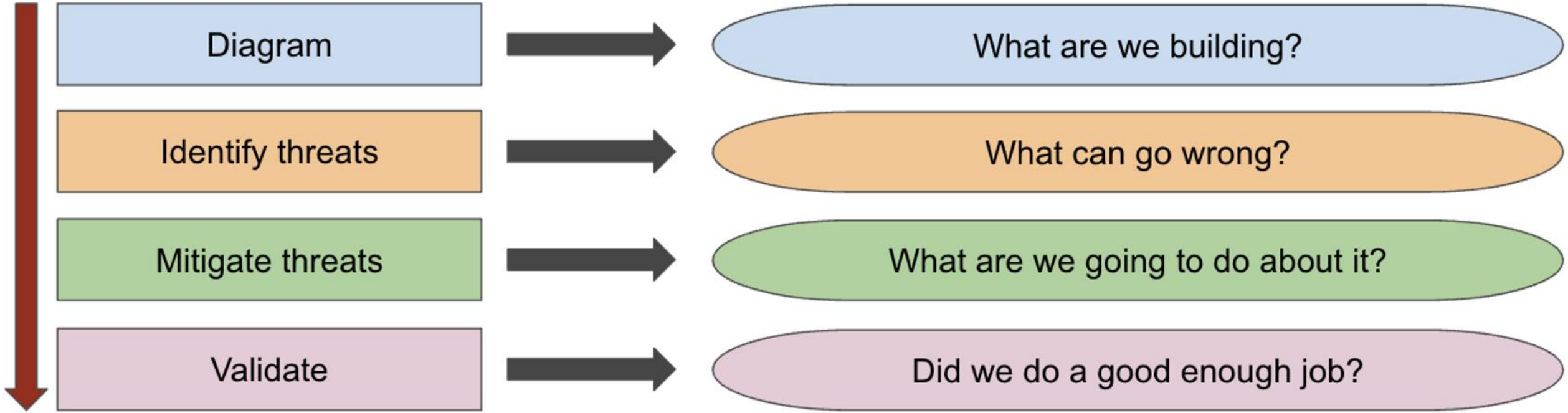
Security throughout development leads to robust and trustworthy software products.

MCSB—controls for DevOps security



https://learn.microsoft.com/en-us/security/benchmark/azure/mcsb-devops-security

Threat Modeling



https://learn.microsoft.com/en-us/training/paths/tm-threat-modeling-fundamentals/

Threat Modeling Frameworks

Framework	Core Focus
STRIDE	General Security
PASTA	Risk-Centric
LINDDUN	Privacy
OCTAVE	Organizational Risk
Trike	System Modeling
VAST	Agile Development

Threat Modeling Methodology

STRIDE

STRIDE is a threat modeling methodology developed by Microsoft to help identify security threats across various dimensions. The acronym stands for:

- **Spoofing,**
- **Tampering,**
- **Repudiation,**
- **Information Disclosure,**
- **Denial of Service,**
- **Elevation of Privilege.**

Each category represents a type of threat that could be posed to a system. By analyzing each component of a system against these six threat types, security teams can systematically identify vulnerabilities and potential attacks.

STRIDE is particularly useful in the early stages of software development, as it allows teams to anticipate and mitigate risks before they become critical. The process involves identifying system entities, defining data flows, and then mapping potential threats to these flows.

Threat Modeling Methodology - STRIDE

1. S - Spoofing Identity

Description: An attacker pretends to be someone or something they are not in order to gain unauthorized access to systems or data.
Example: Using a stolen username and password to log in to a system.
Security Property required: Authentication

2. T - Tampering with Data/System Configurations

Description: Unauthorized modification of data, either at rest or in transit, to alter its meaning or functionality.
Example: Altering the content of a file or a database record to inject malicious code or modify transaction details.
Security Property required: Integrity

3. R - Repudiation

Description: Actions that can be performed in such a way that the actor can deny having performed them. This usually happens when there are insufficient mechanisms in place to prove the occurrence of an action.
Example: A user denying that they made a purchase or performed a transaction because there are no logs or evidence to prove otherwise.
Security Property required: Non-repudiation

Threat Modeling Methodology - Stride

4. I - Information Disclosure

Description: Unauthorized access to or exposure of sensitive information.
Example: An attacker gains access to personal identifiable information (PII) stored in a database due to improper access controls or lack of encryption.
Security Property required: Confidentiality

5. D - Denial of Service (DoS)

Description: Attacks aimed at making a system or service unavailable to its intended users.
Example: Flooding a website with traffic to overwhelm the server and make it unavailable to legitimate users.
Security Property required: Availability

6. E - Elevation of Privilege

Description: A user or attacker gains higher-level permissions than they are authorized to have, allowing them to perform restricted actions.
Example: Exploiting a vulnerability in the system to gain administrative access and perform privileged operations.
Security Property required: Authorization

Threat Modeling Methodology for AI challenges

STRIDE: Strengths and Weaknesses for Agentic AI

- **Strengths:** STRIDE provides a solid foundation for identifying common security vulnerabilities relevant to AI agents, such as data tampering or denial-of-service attacks. It's relatively easy to understand and apply, making it a good starting point for many projects.
- **Weaknesses:** While STRIDE is a good starting point, it doesn't address the unique challenges of AI agents, such as adversarial attacks, or risks due to their unpredictable learning and decision-making processes.
- **Specific AI Gaps:** STRIDE lacks the necessary scope to address threats unique to AI, such as adversarial machine learning, data poisoning, and struggles to model the dynamic, autonomous behaviors of AI agents. It does not explicitly consider the impact of multiple AI agents interacting within an ecosystem.
- **Applicability:** STRIDE can be used as a starting point, but it needs significant augmentation to be effective for Agentic AI, requiring added AI-specific categories or adapting existing ones to reflect new risks.

https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro

Threat Modeling Methodology for AI

To fill these gaps, CSA introduced **MAESTRO (Multi-Agent Environment, Security, Threat Risk, and Outcome)**, a framework built for Agentic AI.

MAESTRO's Principles

•**Extended Security Categories:** We're expanding traditional categories like STRIDE, PASTA, and LINDDUN with AI-specific considerations. For example:

•**Multi-Agent and Environment Focus:** Explicitly considering the interactions between agents and their environment. A self-driving car must be aware of other cars, objects, and weather conditions.

•**Layered Security:** Security isn't a single layer, but a property that must be built into each layer of the agentic architecture.

•**AI-Specific Threats:** Addressing threats arising from AI, especially adversarial ML and autonomy-related risks.

•**Risk-Based Approach:** Prioritizing threats based on likelihood and impact within the agent's context.

•**Continuous Monitoring and Adaptation:** Ongoing monitoring, threat intelligence, and model updates to address the evolving nature of AI and threats.

https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro

Threat Modeling Methodology for AI

To fill these gaps, CSA introduced **MAESTRO (Multi-Agent Environment, Security, Threat Risk, and Outcome)**, a framework built for Agentic AI.

MAESTRO's Principles

•**Extended Security Categories:** We're expanding traditional categories like STRIDE, PASTA, and LINDDUN with AI-specific considerations. For example:

•**Multi-Agent and Environment Focus:** Explicitly considering the interactions between agents and their environment. A self-driving car must be aware of other cars, objects, and weather conditions.

•**Layered Security:** Security isn't a single layer, but a property that must be built into each layer of the agentic architecture.

•**AI-Specific Threats:** Addressing threats arising from AI, especially adversarial ML and autonomy-related risks.

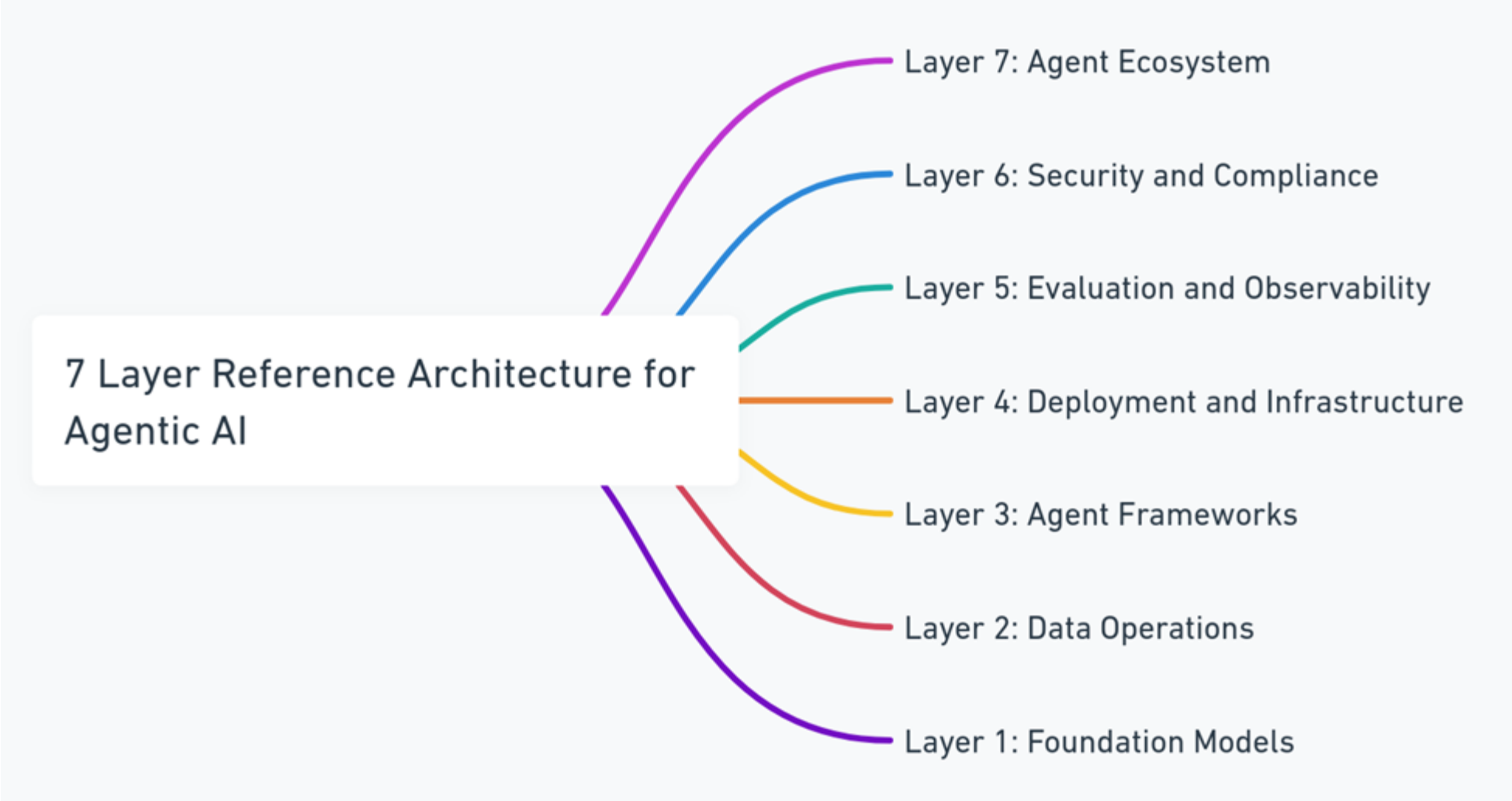
•**Risk-Based Approach:** Prioritizing threats based on likelihood and impact within the agent's context.

•**Continuous Monitoring and Adaptation:** Ongoing monitoring, threat intelligence, and model updates to address the evolving nature of AI and threats.

https://github.com/CloudSecurityAlliance/MAESTRO

https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro

Threat Modeling Methodology for AI



https://cloudsecurityalliance.org/blog/2025/02/06/agentic-ai-threat-modeling-framework-maestro

Multicloud DevOps

Multicloud DevOps platforms

GitHub

The **Community Hub**. Owned by Microsoft, it is the home of open source and has the largest developer network in the world. Best for Open-source projects, startups, and teams prioritizing developer recruitment/familiarity.

GitLab

The **All-in-One**. A single application for the entire DevSecOps lifecycle (planning, SCM, CI/CD, security). Best for Enterprises needing strict governance, self-hosting, and built-in security scanning.

Azure DevOps

The **Enterprise Workhorse**. Deeply integrated with the Azure cloud and the Microsoft 365 ecosystem. Large corporations already using Azure/Active Directory and needing robust project management (Boards).

Bitbucket

The **Atlassian Native**. Seamlessly connects with Jira and Confluence. It is part of the Atlassian "Inner Loop.". Best for business teams already living in Jira who want tight "issue-to-code" traceability.

Multicloud DevOps platforms

Platform	Primary "Flavor"	Best For...
GitHub	The **Community Hub**. Owned by Microsoft, it is the home of open source and has the largest developer network in the world.	Open-source projects, startups, and teams prioritizing developer recruitment/familiarity.
GitLab	The **All-in-One**. A single application for the entire DevSecOps lifecycle (planning, SCM, CI/CD, security).	Enterprises needing strict governance, self-hosting, and built-in security scanning.
Azure DevOps	The **Enterprise Workhorse**. Deeply integrated with the Azure cloud and the Microsoft 365 ecosystem.	Large corporations already using Azure/Active Directory and needing robust project management (Boards).
Bitbucket	The **Atlassian Native**. Seamlessly connects with Jira and Confluence. It is part of the Atlassian "Inner Loop."	Business teams already living in Jira who want tight "issue-to-code" traceability.

Multicloud DevOps platforms

Market share in version control is difficult to measure because it's split between **Public/SaaS** (where GitHub dominates) and **On-Premise/Self-Hosted** (where GitLab and Bitbucket are much stronger).
According to recent 2025–2026 market intelligence reports:

- **GitHub:** Holds roughly **31% to 45%** of the global version control market. It is the undisputed leader in "developer mindshare," hosting over 100 million developers.
- **GitLab:** Commands approximately **18% to 22%** market share. It is the leader in the self-hosted segment, often preferred by regulated industries (Finance/Healthcare) for data residency.
- **Bitbucket:** Occupies about **10% to 15%** of the market. Its user base is concentrated within the Atlassian ecosystem, favored by mid-market companies.
- **Azure DevOps:** While specific "market share" percentages are lower for its *repository* service specifically, it is a dominant force in the broader **DevOps Platform** category, especially among Fortune 500 companies.

DevOps platforms - alternatives

Platform	Core Strength	Security Focus
Harness	AI-Powered Guardrails	Automated Vulnerability Correlation
AWS CodeSuite	Deep AWS Integration	Hardware-level Encryption (KMS)
CircleCI	Pipeline Speed	Fast, Ephemeral Build Environments
Codefresh	GitOps / Kubernetes	No-Credential Production Access
JFrog	Binary Management	Comprehensive Software Bill of Materials (SBOM)

DevSecOps native protection

Platform	SCA (Dependencies)	SAST (Static Code)	DAST (Runtime/Web)	Secret Scanning
GitHub	**Dependabot**	**CodeQL**	No (Requires 3rd party)	**Push Protection**
GitLab	**Dependency Scanning**	**Advanced SAST**	**GitLab DAST**	**Secret Detection**
Azure DevOps	**Advanced Security**	**CodeQL (via GHAS)**	No (Requires 3rd party)	**Secret Scanning**
Bitbucket	**Bitbucket Security**	**Code Insights**	No (Requires 3rd party)	**Secret Scanning**

GHAS – GitHub Advanced Security

Best Practices for DevSecOps

Challenges and How to Overcome Them

Adopting DevSecOps isn’t without hurdles.

Common challenges include:

- **Resistance to Change**: Developers may resist additional security steps. Address this through training and by demonstrating the value of early vulnerability detection.
- **Tool Overload**: Too many tools can overwhelm teams. Prioritize tools that integrate seamlessly with existing workflows.
- **Skill Gaps**: Not all developers are security experts. Invest in ongoing training and pair developers with security champions.

Implementing Continuous Security Assessment

Regular Security Assessments

Conducting regular security assessments ensures early detection of potential vulnerabilities in the development lifecycle.

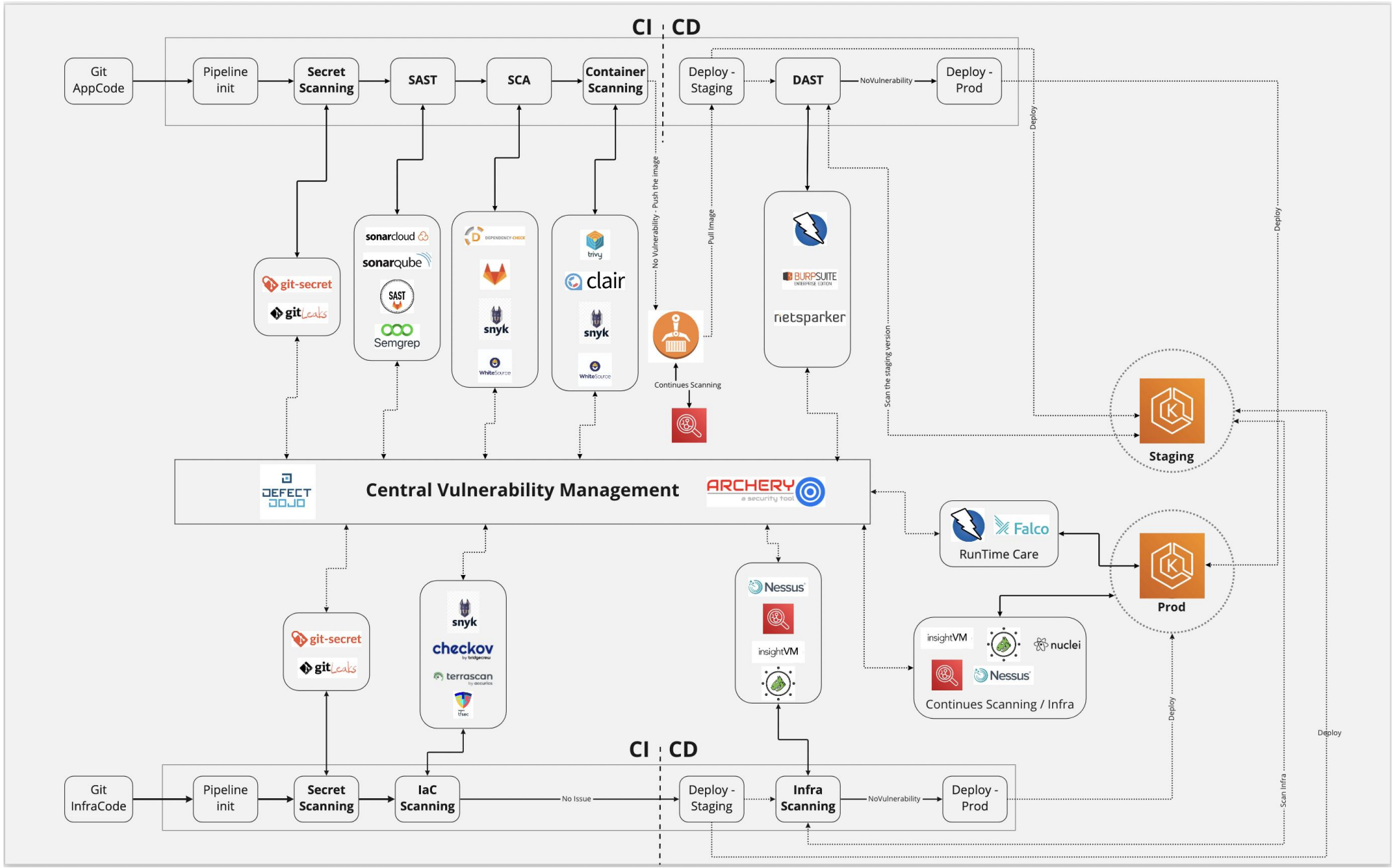
Automated Vulnerability Detection

Automation enables continuous vulnerability scanning, reducing manual effort and increasing assessment frequency.

Integrating Security in Development

Embedding security assessment throughout development maintains protection and reduces risk of breaches.





https://owasp.org/www-project-devsecops-guideline/latest/index

Role-Based Access Control and Secrets Management

Precise Access Rights

Role-based access control ensures users have only necessary permissions, reducing risks of unauthorized access.

Secrets Management

Securely managing secrets like passwords and keys prevents data leaks and strengthens security in DevOps.

Ongoing Training and Collaboration Between Development, Security, and Operations Teams

Shared Responsibility Culture

Encouraging a culture of shared responsibility enhances security by involving all teams in proactive measures.

Continuous Training

Regular training sessions improve security awareness and keep teams updated on best practices.

Effective Communication

Open communication channels between development, security, and operations improve coordination and response times.



Conclusion

Security Integration

Defender for Cloud embeds security directly into DevOps workflows to ensure continuous protection.

Risk Mitigation

It helps mitigate risks by identifying vulnerabilities early in the software delivery process.

Compliance and Efficiency

Enables compliant and efficient software delivery in modern DevSecOps environments.

Dziękuję.
Pytania?